

Projet d'actuariat

BAILLET Benjamin & DJEIGO Yannick

Sommaire

1	Determiner la fonction de probabilité de S	2
2	Comparer la loi de S obtenue par:	3
2.1	Simulation	3
2.2	Algorithme de Panjer	5
2.3	Convolution de la loi de X	7
3	Les fonctions de probabilité de S réunies dans une même fenêtre graphique	9
4	Comparer l'espérance de S obtenue par simulation et par l'algorithme de Panjer	10
5	Comparer le quantile d'ordre 0,05 obtenu par simulation et par l'algorithme de Panjer	11
6	Comparer la Value-at-Risk 5% obtenue par simulation et par l'algorithme de Panjer	12

Sur un modèle collectif, on considère pour loi primaire :

$$N \sim \mathcal{P}(\lambda_1)$$

et pour loi secondaire :

$$X \sim \mathcal{E}(\lambda_2)$$

avec $\lambda_1 = 10$ et $\lambda_2 = \frac{1}{100}$.

Donc on a:

$$P(N = k) = \frac{\lambda_1^k e^{-\lambda_1}}{k!} = \frac{10^k e^{-10}}{k!}, \quad k \in \mathbb{N}$$

$$f_X(X) = \lambda_2 e^{-\lambda_2 x} = \frac{1}{100} e^{-\frac{x}{100}}, \quad x \geq 0$$

En effet, soient $X_1, \dots, X_N$, N variables aléatoires réelles indépendantes et identiquement nulles (iid), de même loi exponentielle de paramètre λ_2 . La loi de la somme $X_1 + \dots + X_N$ est la loi $\Gamma(n, \lambda_2)$.

Donc on définit S par la formule suivante:

$$S = \sum_{i=1}^N X_i \sim \Gamma(n, \lambda_2)$$

où on a: X_i : coût du sinistre i N : nombre aléatoire de sinistres S : charge du contrat (coût total des sinistres)

1 Déterminer la fonction de probabilité de S

$$f_S(s) = \sum_{k=0}^{+\infty} P_k \times \left(f_{\Gamma(k, \lambda_2)}^{*k} \right)(s)$$

On peut réécrire de cette manière:

$$f_S(s) = \sum_{k=0}^{+\infty} P_k \times \left(f_{\Gamma(k, \lambda_2)} * \dots * f_{\Gamma(k, \lambda_2)} \right)(s)$$

Or rappelons que:

$$P_0 = \mathbb{P}(s = 0) = \mathbb{P}(N = 0)$$

On sait également que:

$$\Gamma(k) = (k-1)! \quad \text{pour } k \in \mathbb{N}$$

De plus par convention:

$$f_{\Gamma(k, \lambda_2)}^{*0}(s) = \mathbb{1}_{s=0}$$

Donc on a:

$$f_S(s) = \mathbb{P}(N = 0) * \mathbb{1}_{s=0} + \sum_{k=1}^{+\infty} P_k \times f_{\Gamma(k, \lambda_2)}(s) \mathbb{1}_{s>0}$$

$$\begin{aligned}
&= \mathbb{P}(N = 0) \mathbb{1}_{s=0} + \sum_{k=1}^{+\infty} \mathbb{P}(N = k) \times \frac{\lambda_2^k s^{k-1} e^{-\lambda_2 s}}{\Gamma(k)} \mathbb{1}_{s>0} \\
&= \mathbb{P}(N = 0) \mathbb{1}_{s=0} + \sum_{k=1}^{+\infty} \mathbb{P}(N = k) \times \frac{\lambda_2^k s^{k-1} e^{-\lambda_2 s}}{(k-1)!} \mathbb{1}_{s>0} \\
&= \frac{\lambda_1^0}{0!} e^{-\lambda_1} \mathbb{1}_{s=0} + \sum_{k=1}^{+\infty} \frac{\lambda_1^k}{k!} e^{-\lambda_1} \times \frac{\lambda_2^k s^{k-1} e^{-\lambda_2 s}}{(k-1)!} \mathbb{1}_{s>0}
\end{aligned}$$

Donc finalement voici la fonction de probabilité de S:

$$f_S(s) = e^{-10} \mathbb{1}_{s=0} + \sum_{k=1}^{+\infty} \frac{10^k}{k!} e^{-10} \times \frac{\left(\frac{1}{100}\right)^k s^{k-1} e^{-\frac{s}{100}}}{(k-1)!} \mathbb{1}_{s>0}$$

On peut réécrire la fonction de probabilité de S de cette manière:

$$f_S(s) = \begin{cases} e^{-10} & \text{si } s = 0, \\ e^{-10} e^{-\frac{s}{100}} \sum_{k=1}^{+\infty} \frac{10^k \left(\frac{1}{100}\right)^k s^{k-1}}{k!(k-1)!} & \text{si } s > 0. \end{cases} = \begin{cases} e^{-10} & \text{si } s = 0, \\ e^{-10} e^{-\frac{s}{100}} \sum_{k=1}^{+\infty} \frac{(0.1)^k s^{k-1}}{k!(k-1)!} & \text{si } s > 0. \end{cases}$$

2 Comparer la loi de S obtenue par:

2.1 Simulation

Nous simulons la somme $S = \sum_{i=1}^N X_i$, où $N \sim \mathcal{P}(10)$ (distribution de Poisson) et $X_i \sim \mathcal{E}\left(\frac{1}{100}\right)$ (distribution exponentielle).

```

# Nombre de répétitions de la simulation
nombre_repetitions <- 10000

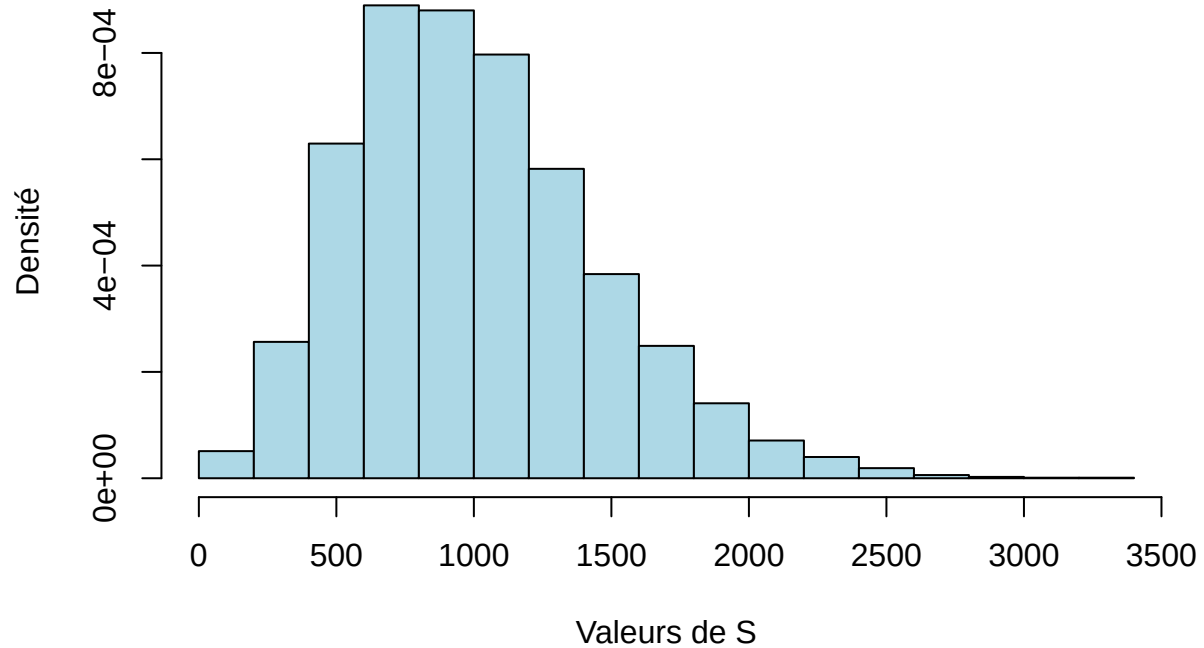
# Simuler le nombre d'observations N à partir d'une loi de Poisson pour toutes les répétitions
N <- rpois(nombre_repetitions, lambda = 10)

# Simuler directement tous les échantillons exponentiels pour chaque répétition
S_sim <- sapply(N, function(n) sum(rexp(n, rate = 1/100)))

# Tracer l'histogramme des valeurs simulées de S
hist(S_sim, freq = FALSE, col = "lightblue", main = "Histogramme de S par simulation",
     xlab = "Valeurs de S", ylab = "Densité")

```

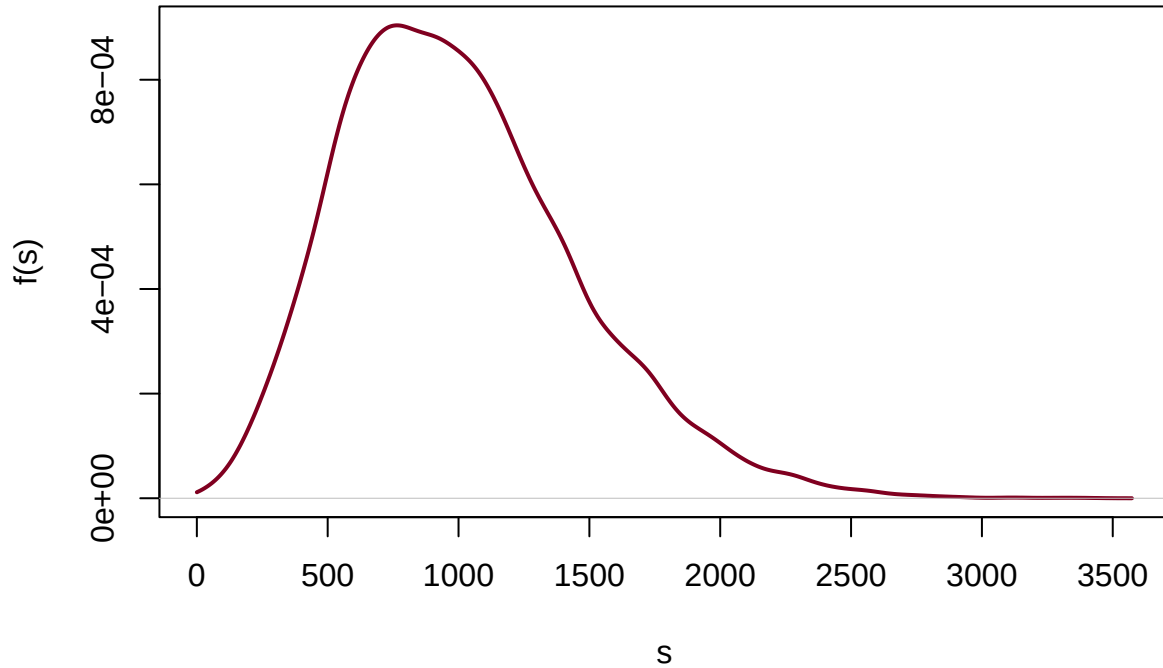
Histogramme de S par simulation



```
# Tracer la fonction de densité obtenue par simulation
densite_sim <- density(S_sim, from = 0)

plot(densite_sim,
     main = "Densité obtenue par simulation",
     xlab = "s",
     ylab = "f(s)",
     col = "#800020", # Rouge bordeaux
     lwd = 2)
```

Densité obtenue par simulation



2.2 Algorithme de Panjer

L'algorithme de Panjer est utilisé pour calculer la distribution de S de façon récursive à partir des lois primaires et secondaires

2.2.1 Loi de Panjer

On définit la loi de Panjer comme suit:

N suit la loi de Panjer de paramètres a ($a < 1$) et b ($a+b > 0$) noté $N \sim PA(a, b)$

Si $\forall k \in \mathbb{N} \setminus \{0\}; p_k = (a + \frac{b}{k})p_{k-1}$.

- Si $a = 0 \Rightarrow PA(a = 0, b) = \mathcal{P}(b)$
- Si $a < 0$ et $n = \frac{-b}{a} - 1 \in \mathbb{N} \Rightarrow PA(a, b) = b(n, p)$ avec $p = \frac{-a}{1-a}$
- Si $0 < a < 1 \Rightarrow PA(a, b) = BN(r, p)$ avec $r = \frac{b}{a} + 1$ et $p = 1 - a$

2.2.2 Algorithme de Panjer

On suppose initialement que la loi primaire $N \sim PA(a, b)$ et la loi secondaire X est à valeurs dans $\mathbb{N}$.

On note $\forall j \in \mathbb{N}$:

$p_j = \mathbb{P}(N = j)$, $q_j = \mathbb{P}(X = j)$, $r_j = \mathbb{P}(S = j)$ On définit alors:

$$r_0 = \sum_{j=0}^{+\infty} q_0^j p_j$$

$$\forall j \geq 1, \quad r_j = \frac{1}{1 - a q_0} \sum_{i=1}^j \left(a + \frac{b \cdot i}{j}\right) q_i r_{j-i}$$

Selon l'exercice, $N \sim \mathcal{P}(10)$ alors on a $a = 0$ et $b = 10$.

Ainsi,

```
# Mémorisation : stocke les résultats déjà calculés
memo <- list()

# Fonction récursive avec mémoïsation
panjer_S <- function(s) {
  lambda1 <- 10      # paramètre de la loi de Poisson
  lambda2 <- 1/100    # paramètre de la loi exponentielle

  # Cas de base : s <= 0
  if (s <= 0) {
    return(sum(0^(0:300) * dpois(0:300, lambda = lambda1)))
  }

  s <- as.integer(floor(s) + 1) # Discrétisation de s

  # Vérifie si la valeur a déjà été calculée
  if (length(memo) >= s) {
    return(memo[[s]])
  }

  # Calcul de rj
  rj <- sum((lambda1 * (1:s) / s) * (pexp(1:s, lambda2) - pexp(0:(s-1), lambda2)) * sapply(1:s, function(x) {
    # Stocke le résultat dans la mémorisation
    memo[[x]] <- rj
    return(rj)
  }

  # Nombre de valeurs calculées
  nb_val1 <- 10001 # Nombre de points pour lesquels on va implémenter l'algorithme de Panjer

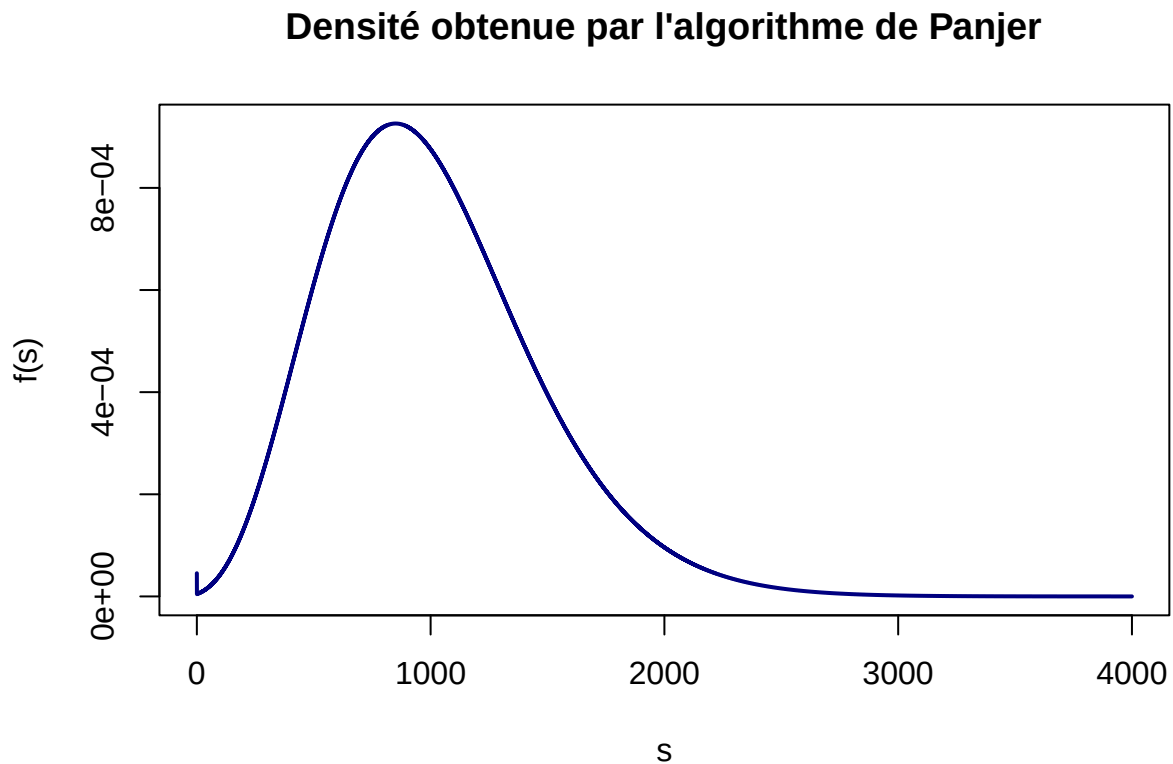
  # Initialiser un vecteur pour stocker les valeurs de S
  S_panjer <- numeric(nb_val1)

  # Calcul des valeurs de la densité de S par la fonction de convolution
  s1 <- seq(0, 4000, length.out = nb_val1) # Calcul de nb_val1 valeurs de f_S entre 0 et 4000

  # Remplissage de S_panjer
  S_panjer <- sapply(s1, panjer_S)

  # La densité de Panjer est maintenant stockée dans S_panjer
  densite_panjer <- S_panjer
```

```
# Densité obtenue par algorithme de Panjer
plot(s1, densite_panjer,
     type = "l",
     main = "Densité obtenue par l'algorithme de Panjer",
     xlab = "s",
     ylab = "f(s)",
     col = "#000080",
     lwd = 2)
```



2.3 Convolution de la loi de X

On effectue une convolution des distributions pour obtenir la distribution de S.

```
# Fonction de convolution
f_S = function(s){ # On reprend la fonction trouvée dans la question 1)
  if (s > 0){
    n = 500 # On prends des valeurs de n entre 1 et 500 (n>500 est négligeable dans le produit d'une po
    return(sum( dpois(1:n, lambda = 10) * dgamma(s, shape = 1:n, rate = 1/100)) )
  } else { # Pour s = 0
    return(exp(-10))
  }
}

# Nombre de valeurs calculées
```

```

nb_val2 = 10001 # Nombre de points pour lesquels on va calculer l'image par la fonction f_S

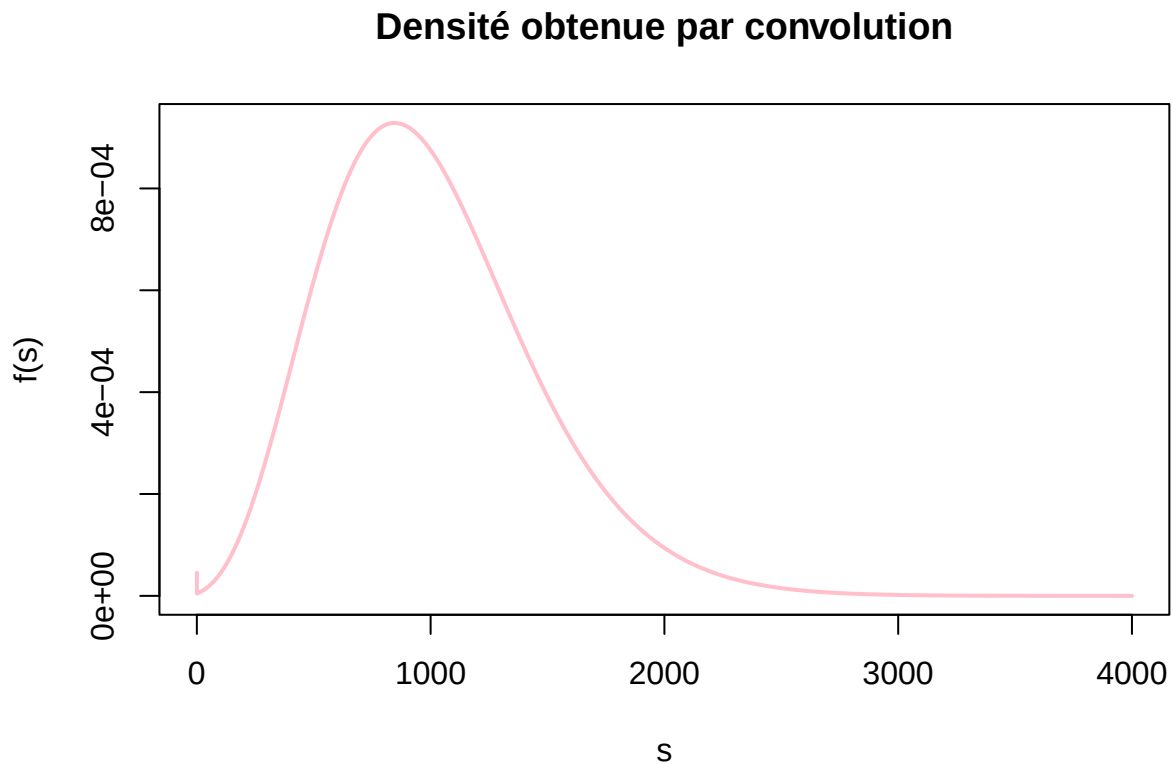
# Initialiser un vecteur pour stocker les valeur de S
S_conv = numeric(nb_val2)

# Calcul des valeurs de la densité de S par la fonction de convolution
s2 = seq(0, 4000, length.out = nb_val2) # On va calculer nb_vals valeurs de f_S entre 0 et 4000
for (i in 1:nb_val2) {
  S_conv[i] = f_S(s2[i])
}

# Fonction de densité
densite_conv = S_conv

# Densité obtenue par convolution
plot(s2, densite_conv,
     type = "l",
     main = "Densité obtenue par convolution",
     xlab = "s",
     ylab = "f(s)",
     col = "pink", # Rose
     lwd = 2)

```



3 Les fonctions de probabilité de S réunies dans une même fenêtre graphique

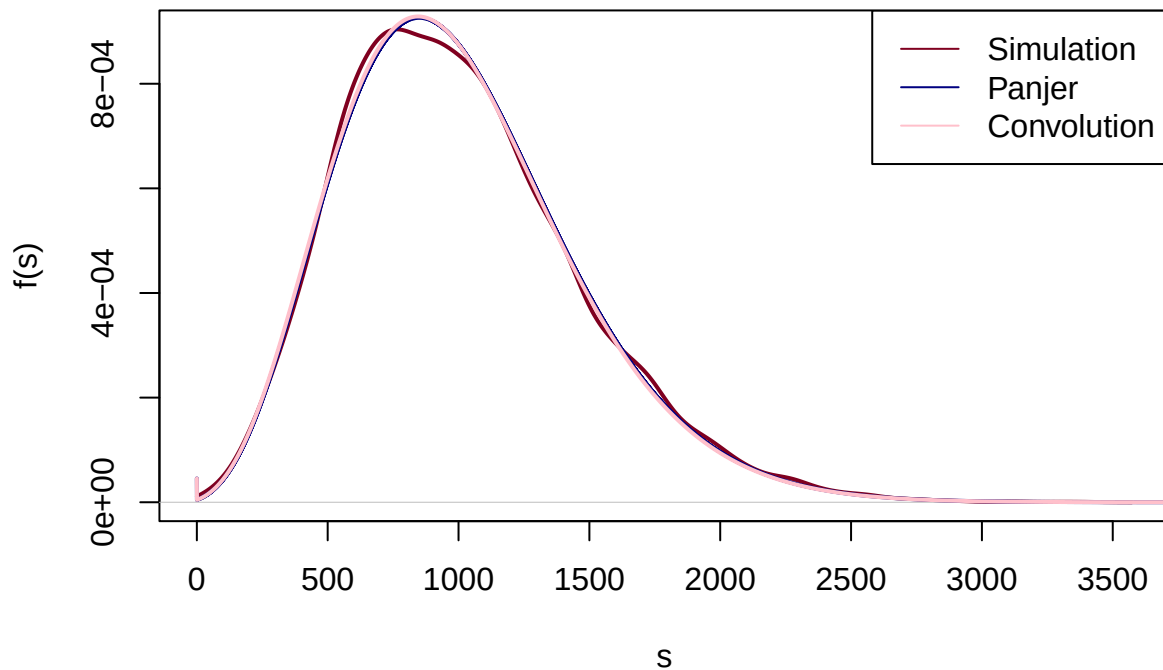
```
# Densité obtenue par simulation
plot(densite_sim,
     main = "Comparaison des 3 densités",
     xlab = "s",
     ylab = "f(s)",
     col = "#800020", # Rouge bordeaux
     lwd = 2)

# Densité obtenue par l'algorithme de Panjer
lines(s1, densite_panjer,
      col = "#000080", # Bleu
      lwd = 2)

# Densité obtenue par convolution
lines(s2, densite_conv,
      col = "pink", # Rose
      lwd = 2)

# Légende
legend("topright",
      legend = c("Simulation", "Panjer", "Convolution"),
      col = c("#800020", "#000080", "pink"),
      lty = rep(1))
```

Comparaison des 3 densités



4 Comparer l'espérance de S obtenue par simulation et par l'algorithme de Panjer

Méthode 1: par calcul explicite

Selon le cours on sait que

$$E(S) = E(N) \times E(X)$$

```
# Nombre de répétitions pour la simulation
nombre_repetitions <- 10000

# Simuler la somme S
S_sim <- numeric(nombre_repetitions)

for (i in 1:nombre_repetitions) {
  N <- rpois(1, lambda = 10) # Simuler N suivant une loi de Poisson
  X <- rexp(N, rate = 1/100) # Simuler N échantillons de la loi exponentielle
  S_sim[i] <- sum(X)         # Stocker la somme
}

# Calcul de l'espérance par simulation
E_sim <- mean(S_sim) # Calculer la moyenne des résultats

# Espérance analytique
```

```

E_N <- 10          # Espérance d'une loi de Poisson avec lambda = 10
E_X <- 100         # Espérance d'une loi exponentielle avec rate = 1/100
E_pan <- E_N * E_X  # Espérance par l'algorithme de Panjer

```

```

# Affichage des résultats
cat("L'espérance de S obtenue par simulation :", E_sim, "\n")

```

```

## L'espérance de S obtenue par simulation : 995.7616

```

```

cat("L'espérance de S obtenue par l'algorithme de Panjer :", E_pan, "\n")

```

```

## L'espérance de S obtenue par l'algorithme de Panjer : 1000

```

Méthode 2: on utilise la méthode des rectangles

```

# Calcul de l'espérance par simulation
E_sim <- sum(densite_sim$x * densite_sim$y * diff(densite_sim$x))

```

```

# Calcul de l'espérance par l'algorithme de Panjer
E_pan <- sum(s1 * densite_panjer * diff(s1))

```

```

# Affichage des résultats
cat("L'espérance de S obtenue par simulation :", E_sim, "\n")

```

```

## L'espérance de S obtenue par simulation : 1007.036

```

```

cat("L'espérance de S obtenue par l'algorithme de Panjer :", E_pan, "\n")

```

```

## L'espérance de S obtenue par l'algorithme de Panjer : 1004.397

```

5 Comparer le quantile d'ordre 0,05 obtenu par simulation et par l'algorithme de Panjer

```

# Quantile par simulation
quant_sim <- quantile(S_sim, probs = 0.05)

```

```

# Quantile par l'algorithme de Panjer

```

```

# Calculer la fonction de répartition cumulée à partir de la densité
cumulative_prob <- cumsum(densite_panjer) * diff(c(0, s1))
quant_panj <- s1[which(cumulative_prob >= 0.05)[1]] # Trouver le quantile

```

```

# Résultat
cat("Le quantile d'ordre 0.05 obtenu par simulation est :", quant_sim, "\n")

```

```

## Le quantile d'ordre 0.05 obtenu par simulation est : 360.303

```

```
cat("Le quantile d'ordre 0.05 obtenu par l'algorithme de Panjer est :", quant_panj, "\n")
```

```
## Le quantile d'ordre 0.05 obtenu par l'algorithme de Panjer est : 362.4
```

6 Comparer la Value-at-Risk 5% obtenue par simulation et par l'algorithme de Panjer

La VaR à 5 % est la perte maximale que l'on ne devrait pas dépasser avec une confiance de 95 %. En général, la VaR est calculée comme $1 - \text{quantile}(0.95)$ pour indiquer la perte plutôt que le gain.

```
## VAR Simulation
```

```
VAR_sim = quantile(S_sim, probs = 0.95)
```

```
VAR_sim = abs(1 - VAR_sim)
```

```
## VAR Panjer
```

```
# On veut trouver l'indice correspondant au quantile d'ordre 0.95
```

```
cumulative_prob <- cumsum(densite_panj * diff(s1))
```

```
index_quantile_0.95 <- which(cumulative_prob >= 0.95)[1]
```

```
# Récupérer la valeur de x correspondant au quantile d'ordre 0.95
```

```
VAR_panj <- abs(s1[index_quantile_0.95])
```

```
# Résultat
```

```
cat("La Value-at-Risk (VaR) à 5 % obtenue par simulation est :", VAR_sim, "\n")
```

```
## La Value-at-Risk (VaR) à 5 % obtenue par simulation est : 1788.22
```

```
cat("La Value-at-Risk (VaR) à 5 % obtenue par l'algorithme de Panjer est :", VAR_panj, "\n")
```

```
## La Value-at-Risk (VaR) à 5 % obtenue par l'algorithme de Panjer est : 1818.8
```